

AGENTS AND PARALLEL WORK

Create custom subagents

Create and use specialized AI subagents in Claude Code for task-specific workflows and improved context management.

Subagents are specialized AI assistants that handle specific types of tasks. Use one when a side task would flood your main conversation with search results, logs, or file contents you won't reference again: the subagent does that work in its own context and returns only the summary. Define a custom subagent when you keep spawning the same kind of worker with the same instructions.

Each subagent runs in its own context window with a custom system prompt, specific tool access, and independent permissions. When Claude encounters a task that matches a subagent's description, it delegates to that subagent, which works independently and returns results. To see the context savings in practice, the [context window visualization](#) walks through a session where a subagent handles research in its own separate window.

❗ Subagents work within a single session. To run many independent sessions in parallel and monitor them from one place, see [background agents](#). For sessions that communicate with each other, see [agent teams](#).

Subagents help you:

- **Preserve context** by keeping exploration and implementation out of your main conversation
- **Enforce constraints** by limiting which tools a subagent can use
- **Reuse configurations** across projects with user-level subagents
- **Specialize behavior** with focused system prompts for specific domains
- **Control costs** by routing tasks to faster, cheaper models like Haiku

Claude uses each subagent's description to decide when to delegate tasks. When you create a subagent, write a clear description so Claude knows when to use it.

Claude Code includes several built-in subagents like **Explore**, **Plan**, and **general-purpose**. You can also create custom subagents to handle specific tasks.

Built-in subagents

Claude Code includes built-in subagents that Claude automatically uses when appropriate. Each inherits the parent conversation's permissions with additional tool restrictions.

Explore and Plan skip your CLAUDE.md files and the parent session's git status to keep research fast and inexpensive. Every other built-in and **custom subagent** loads both. For the full breakdown of what reaches a subagent, see **what loads at startup**.

Explore

A fast, read-only agent optimized for searching and analyzing codebases.

- **Model:** Haiku, which is fast and low-latency
- **Tools:** read-only tools; Write and Edit are denied
- **Purpose:** file discovery, code search, codebase exploration

Claude delegates to Explore when it needs to search or understand a codebase without making changes. This keeps exploration results out of your main conversation context. When invoking Explore, Claude specifies a thoroughness level: **quick** for targeted lookups, **medium** for balanced exploration, or **very thorough** for comprehensive analysis.

Plan

A research agent used during **plan mode** to gather context before presenting a plan.

- **Model:** inherits from the main conversation
- **Tools:** read-only tools; Write and Edit are denied
- **Purpose:** codebase research for planning

When you're in plan mode and Claude needs to understand your codebase, it delegates research to the Plan subagent so that exploration output stays in a separate context window while the main conversation remains read-only.

General-purpose

A capable agent for complex, multi-step tasks that require both exploration and action.

- **Model:** inherits from the main conversation
- **Tools:** all tools
- **Purpose:** complex research, multi-step operations, code modifications

Claude delegates to general-purpose when the task requires both exploration and modification, complex reasoning to interpret results, or multiple dependent steps.

Other

Claude Code includes additional helper agents for specific tasks. These are typically invoked automatically, so you don’t need to use them directly.

Agent	Model	When Claude uses it
statusline-setup	Sonnet	When you run <code>/statusline</code> to configure your status line
claude-code-guide	Haiku	When you ask questions about Claude Code features

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after
code changes.",
    "prompt": "You are a senior code reviewer. Focus on code
quality, security, and best practices.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  },
  "debugger": {
    "description": "Debugging specialist for errors and test
failures.",
    "prompt": "You are an expert debugger. Analyze errors,
identify root causes, and provide fixes."
  }
}'
```

```

claude --agents '@'
{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after
code changes.",
    "prompt": "You are a senior code reviewer. Focus on code
quality, security, and best practices.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  },
  "debugger": {
    "description": "Debugging specialist for errors and test
failures.",
    "prompt": "You are an expert debugger. Analyze errors,
identify root causes, and provide fixes."
  }
}
'@

```

Built-in subagents are always registered in interactive sessions. To restrict them:

- To block a specific built-in type, add it to `permissions.deny` as shown in [Disable specific subagents](#).
- To prevent Claude from delegating to any subagent, deny the `Agent` tool itself with `permissions.deny`.
- In **non-interactive mode** and the **Agent SDK**, set `CLAUDE_AGENT_SDK_DISABLE_BUILTIN_AGENTS=1` to remove all built-in types and supply only your own.

Beyond these built-in subagents, you can create your own with custom prompts, tool restrictions, permission modes, hooks, and skills. The following sections show how to get started and customize subagents.

Quickstart: create your first subagent

Subagents are defined in Markdown files with YAML frontmatter. You can [create them manually](#) or

use the `/agents` command.

This walkthrough guides you through creating a user-level subagent with the `/agents` command. The subagent reviews code and suggests improvements for the codebase.

1 Open the subagents interface

In Claude Code, run:

```
/agents
```

2 Choose a location

Switch to the **Library** tab, select **Create new agent**, then choose **Personal**. This saves the subagent to `~/.claude/agents/` so it's available in all your projects.

3 Generate with Claude

Select **Generate with Claude**. When prompted, describe the subagent:

```
A code improvement agent that scans files and suggests
improvements
for readability, performance, and best practices. It should
explain
each issue, show the current code, and provide an improved
version.
```

Claude generates the identifier, description, and system prompt for you.

4 Select tools

For a read-only reviewer, deselect everything except **Read-only tools**. If you keep all tools selected, the subagent inherits all tools available to the main conversation.

5 Select model

Choose which model the subagent uses. For this example agent, select **Sonnet**, which balances capability and speed for analyzing code patterns.

6 Choose a color

Pick a background color for the subagent. This helps you identify which subagent is running in the UI.

7 Configure memory

Select **User scope** to give the subagent a **persistent memory directory** at `~/.claude/agent-memory/`. The subagent uses this to accumulate insights across conversations, such as codebase patterns and recurring issues. Select **None** if you don't want the subagent to persist learnings.

8 Save and try it out

Review the configuration summary. Press `s` or `Enter` to save, or press `e` to save and edit the file in your editor. The subagent is available immediately. Try it:

```
Use the code-improver agent to suggest improvements in this
project
```

Claude delegates to your new subagent, which scans the codebase and returns improvement suggestions.

You now have a subagent you can use in any project on your machine to analyze codebases and suggest improvements.

You can also create subagents manually as Markdown files, define them via CLI flags, or distribute them through plugins. The following sections cover all configuration options.

Configure subagents

Use the `/agents` command

The `/agents` command opens a tabbed interface for managing subagents. The **Running** tab lists live

and recently finished subagents and lets you open or stop them. The **Library** tab lets you:

- View all available subagents (built-in, user, project, and plugin)
- Create new subagents with guided setup or Claude generation
- Edit existing subagent configuration and tool access
- Delete custom subagents
- See which subagents are active when duplicates exist

This is the recommended way to create and manage subagents. For manual creation or automation, you can also add subagent files directly.

Choose the subagent scope

Subagents are Markdown files with YAML frontmatter. Store them in different locations depending on scope. When multiple subagents share the same name, Claude Code uses the one from the higher-priority location.

Location	Scope	Priority	How to create
Managed settings	Organization-wide	1 (highest)	Deployed via <u>managed settings</u>
<code>--agents</code> CLI flag	Current session	2	Pass JSON when launching Claude Code
<code>.claude/agents/</code>	Current project	3	Interactive or manual
<code>~/.claude/agents/</code>	All your projects	4	Interactive or manual
Plugin's <code>agents/</code> directory	Where plugin is enabled	5 (lowest)	Installed with <u>plugins</u>

Project subagents (`.claude/agents/`) are ideal for subagents specific to a codebase. Check them into version control so your team can use and improve them collaboratively.

Project subagents are discovered by walking up from the current working directory, so every `.claude/agents/` between there and the repository root is scanned. As of v2.1.178, when more than one of these nested directories defines the same `name` , Claude Code uses the definition closest to the working directory.

Directories added with `--add-dir` are also scanned: a `.claude/agents/` folder inside an added directory loads alongside project subagents. See Additional directories for which other configuration types load from `--add-dir` . To share subagents across projects without `--add-dir` ,

use `~/.claude/agents/` or a **plugin**.

User subagents (`~/.claude/agents/`) are personal subagents available in all your projects.

Claude Code scans `.claude/agents/` and `~/.claude/agents/` recursively, so you can organize definitions into subfolders such as `agents/review/` or `agents/research/`. The subdirectory path doesn't affect how a subagent is identified or invoked, because identity comes only from the `name` frontmatter field. Keep `name` values unique across the whole tree: if two files within one scope declare the same name, Claude Code keeps one and discards the other without warning.

Plugin `agents/` directories are also scanned recursively. Unlike project and user scopes, a subfolder inside a plugin's `agents/` directory becomes part of the **scoped identifier**: a file at `agents/review/security.md` in plugin `my-plugin` registers as `my-plugin:review:security`.

CLI-defined subagents are passed as JSON when launching Claude Code. They exist only for that session and aren't saved to disk, making them useful for quick testing or automation scripts. You can define multiple subagents in a single `--agents` call:

macOS, Linux, WSL

A fast, read-only agent optimized for searching and analyzing codebases.

- **Model:** Haiku, which is fast and low-latency
- **Tools:** read-only tools; Write and Edit are denied
- **Purpose:** file discovery, code search, codebase exploration

Claude delegates to Explore when it needs to search or understand a codebase without making changes. This keeps exploration results out of your main conversation context. When invoking Explore, Claude specifies a thoroughness level: **quick** for targeted lookups, **medium** for balanced exploration, or **very thorough** for comprehensive analysis.

Windows PowerShell

A research agent used during **plan mode** to gather context before presenting a plan.

- **Model:** inherits from the main conversation
- **Tools:** read-only tools; Write and Edit are denied
- **Purpose:** codebase research for planning

When you're in plan mode and Claude needs to understand your codebase, it delegates research to the Plan subagent so that exploration output stays in a separate context window while the main conversation remains read-only.

A capable agent for complex, multi-step tasks that require both exploration and action.

- **Model:** inherits from the main conversation
- **Tools:** all tools
- **Purpose:** complex research, multi-step operations, code modifications

Claude delegates to general-purpose when the task requires both exploration and modification, complex reasoning to interpret results, or multiple dependent steps.

Claude Code includes additional helper agents for specific tasks. These are typically invoked automatically, so you don't need to use them directly.

Agent	Model	When Claude uses it
statusline-setup	Sonnet	When you run <code>/statusline</code> to configure your status line
claude-code-guide	Haiku	When you ask questions about Claude Code features

```
claude --agents '{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after
code changes.",
    "prompt": "You are a senior code reviewer. Focus on code
quality, security, and best practices.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  },
  "debugger": {
    "description": "Debugging specialist for errors and test
failures.",
    "prompt": "You are an expert debugger. Analyze errors,
identify root causes, and provide fixes."
  }
}'
```

```
claude --agents '@'
{
  "code-reviewer": {
    "description": "Expert code reviewer. Use proactively after
code changes.",
    "prompt": "You are a senior code reviewer. Focus on code
quality, security, and best practices.",
    "tools": ["Read", "Grep", "Glob", "Bash"],
    "model": "sonnet"
  },
  "debugger": {
    "description": "Debugging specialist for errors and test
failures.",
    "prompt": "You are an expert debugger. Analyze errors,
identify root causes, and provide fixes."
  }
}
'@
```

The `--agents` flag accepts JSON with the same **frontmatter** fields as file-based subagents: `description`, `prompt`, `tools`, `disallowedTools`, `model`, `permissionMode`, `mcpServers`, `hooks`, `maxTurns`, `skills`, `initialPrompt`, `memory`, `effort`, `background`, `isolation`, and `color`. Use `prompt` for the system prompt, equivalent to the markdown body in file-based subagents.

Managed subagents are deployed by organization administrators. Place markdown files in `.claude/agents/` inside the **managed settings directory**, using the same frontmatter format as project and user subagents. Managed definitions take precedence over project and user subagents with the same name.

Plugin subagents come from **plugins** you've installed. They appear in `/agents` alongside your custom subagents. See the **plugin components reference** for details on creating plugin subagents.

❗ For security reasons, plugin subagents don't support the `hooks`, `mcpServers`, or `permissionMode` frontmatter fields. These fields are ignored when loading agents from a plugin. If you need them, copy the agent file into `.claude/agents/` or `~/.claude/agents/`. You can also add rules to **permissions.allow** in `settings.json` or `settings.local.json`, but these rules apply to the entire session, not only the plugin subagent.

Subagent definitions from any of these scopes are also available to **agent teams**: when spawning a teammate, you can reference a subagent type and the teammate uses its `tools` and `model`, with the definition's body appended to the teammate's system prompt as additional instructions. See **agent teams** for which frontmatter fields apply on that path.

Write subagent files

Subagent files use YAML frontmatter for configuration, followed by the system prompt in Markdown:

❗ Subagents are loaded at session start. If you add or edit a subagent file directly on disk, restart your session to load it. Subagents created through the `/agents` interface take effect immediately without a restart.

```
---
name: code-reviewer
description: Reviews code for quality and best practices
tools: Read, Glob, Grep
model: sonnet
---
```

You are a code reviewer. When invoked, analyze the code and provide specific, actionable feedback on quality, security, and best practices.

The frontmatter defines the subagent’s metadata and configuration. The body becomes the system prompt that guides the subagent’s behavior. Subagents receive only this system prompt plus basic environment details like the working directory, not the full Claude Code system prompt.

A subagent starts in the main conversation’s current working directory. Within a subagent, `cd` commands don’t persist between Bash or PowerShell tool calls and don’t affect the main conversation’s working directory. To give the subagent an isolated copy of the repository instead, set `isolation: worktree`.

Supported frontmatter fields

The following fields can be used in the YAML frontmatter. Only `name` and `description` are required.

Field	Required	Description
<code>name</code>	Yes	Unique identifier using lowercase letters and hyphens. Hooks receive this value as <code>agent_type</code> . The filename doesn’t have to match
<code>description</code>	Yes	When Claude should delegate to this subagent
<code>tools</code>	No	Tools the subagent can use. Inherits all tools if omitted. To preload Skills into context, use the <code>skills</code> field rather than listing <code>Skill</code> here
<code>disallowedTools</code>	No	Tools to deny, removed from inherited or specified list
<code>model</code>	No	Model to use: <code>sonnet</code> , <code>opus</code> , <code>haiku</code> , <code>fable</code> , a full model ID (for example, <code>claude-opus-4-8</code>), or <code>inherit</code> . Defaults to <code>inherit</code>
<code>permissionMode</code>	No	Permission mode: <code>default</code> , <code>acceptEdits</code> , <code>auto</code> , <code>dontAsk</code> , <code>bypassPermissions</code> , or <code>plan</code> . Ignored for plugin subagents

Field	Required	Description
maxTurns	No	Maximum number of agentic turns before the subagent stops
skills	No	Skills to preload into the subagent’s context at startup. The full skill content is injected, not only the description. Subagents can still invoke unlisted project, user, and plugin skills through the Skill tool
mcpServers	No	MCP servers available to this subagent. Each entry is either a server name referencing an already-configured server (e.g., "slack") or an inline definition with the server name as key and a full MCP server config as value. Ignored for plugin subagents
hooks	No	Lifecycle hooks scoped to this subagent. Ignored for plugin subagents
memory	No	Persistent memory scope: user , project , or local . Enables cross-session learning
background	No	Set to true to always run this subagent as a background task . Default: false
effort	No	Effort level when this subagent is active. Overrides the session effort level. Default: inherits from session. Options: low , medium , high , xhigh , max ; available levels depend on the model
isolation	No	Set to worktree to run the subagent in a temporary git worktree , giving it an isolated copy of the repository branched by default from your default branch rather than the parent session’s HEAD . The worktree is automatically cleaned up if the subagent makes no changes
color	No	Display color for the subagent in the task list and transcript. Accepts red , blue , green , yellow , purple , orange , pink , or cyan
initialPrompt	No	Auto-submitted as the first user turn when this agent runs as the main session agent (via --agent or the agent setting). Commands and skills are processed. Prepend to any user-provided prompt

Choose a model

The model field controls which **AI model** the subagent uses:

- Model alias:** use one of the available aliases: sonnet , opus , haiku , or fable
- Full model ID:** use a full model ID such as claude-opus-4-8 or claude-sonnet-4-6 . Accepts the same values as the --model flag
- inherit:** use the same model as the main conversation

- **Omitted:** defaults to `inherit` and uses the same model as the main conversation

When Claude invokes a subagent, it can also pass a `model` parameter for that specific invocation. Claude Code resolves the subagent's model in this order:

1. The `CLAUDE_CODE_SUBAGENT_MODEL` environment variable, if set
2. The per-invocation `model` parameter
3. The subagent definition's `model` frontmatter
4. The main conversation's model

The environment variable, per-invocation parameter, and frontmatter values are checked against your organization's `availableModels` allowlist. A value that resolves to an excluded model is not used and the subagent runs on the inherited model instead.

Control subagent capabilities

You can control what subagents can do through tool access, permission modes, and conditional rules.

Available tools

Subagents inherit the **internal tools** and MCP tools available in the main conversation by default. The following tools depend on the main conversation's UI or session state and are not available to subagents, even when listed in the `tools` field:

- `AskUserQuestion`
- `EnterPlanMode`
- `ExitPlanMode` , unless the subagent's `permissionMode` is `plan`
- `ScheduleWakeup`
- `WaitForMcpServers`

To restrict tools, use either the `tools` field (allowlist) or the `disallowedTools` field (denylist). This example uses `tools` to exclusively allow Read, Grep, Glob, and Bash. The subagent can't edit files, write files, or use any MCP tools:

```
---
name: safe-researcher
description: Research agent with restricted capabilities
tools: Read, Grep, Glob, Bash
---
```

This example uses `disallowedTools` to inherit every tool from the main conversation except Write and Edit. The subagent keeps Bash, MCP tools, and everything else:

```
---
name: no-writes
description: Inherits every tool except file writes
disallowedTools: Write, Edit
---
```

If both are set, `disallowedTools` is applied first, then `tools` is resolved against the remaining pool. A tool listed in both is removed.

Both fields accept MCP server-level patterns in addition to exact tool names: `mcp__<server>` or `mcp__<server>__*` grants or removes every tool from the named server. In `disallowedTools`, `mcp__*` also removes every MCP tool from any server. This example removes every tool from the `github` MCP server while keeping tools from other servers and every built-in tool:

```
---
name: local-only
description: Inherits every tool except those from the github MCP
server
disallowedTools: mcp__github
---
```

Restrict which subagents can be spawned

When an agent runs as the main thread with `claude --agent`, it can spawn subagents using the Agent tool. To restrict which subagent types it can spawn, use `Agent(agent_type)` syntax in the `tools` field.

❗ In version 2.1.63, the Task tool was renamed to Agent. Existing `Task(...)` references in settings and agent definitions still work as aliases.

```
---
name: coordinator
description: Coordinates work across specialized agents
tools: Agent(worker, researcher), Read, Bash
---
```

This is an allowlist: only the `worker` and `researcher` subagents can be spawned. If the agent tries to spawn any other type, the request fails and the agent sees only the allowed types in its prompt. To block specific agents while allowing all others, use `permissions.deny` instead.

To allow spawning any subagent without restrictions, use `Agent` without parentheses:

```
tools: Agent, Read, Bash
```

If `Agent` is omitted from the `tools` list entirely, the agent can't spawn any subagents.

The `Agent(agent_type)` allowlist syntax applies only to an agent running as the main thread with `claude --agent`. In a subagent definition, listing `Agent` in `tools` lets that subagent **spawn nested subagents**, but any type list inside the parentheses is ignored.

Scope MCP servers to a subagent

Use the `mcpServers` field to give a subagent access to **MCP** servers that aren't available in the main conversation. Inline servers defined here are connected when the subagent starts and disconnected when it finishes. String references share the parent session's connection.

❗ The `mcpServers` field applies in both contexts where an agent file can run:

- As a subagent, spawned through the `Agent` tool or an @-mention
- As the main session, launched with `--agent` or the `agent` setting

When the agent is the main session, inline server definitions connect at startup alongside servers from `.mcp.json` and settings files.

Each entry in the list is either an inline server definition or a string referencing an MCP server already configured in your session:

```

---
name: browser-tester
description: Tests features in a real browser using Playwright
mcpServers:
  # Inline definition: scoped to this subagent only
  - playwright:
      type: stdio
      command: npx
      args: ["-y", "@playwright/mcp@latest"]
  # Reference by name: reuses an already-configured server
  - github
---

```

Use the Playwright tools to navigate, screenshot, and interact with pages.

Inline definitions use the same schema as `.mcp.json` server entries (`stdio` , `http` , `sse` , `ws`), keyed by the server name.

To keep an MCP server out of the main conversation entirely and avoid its tool descriptions consuming context there, define it inline here rather than in `.mcp.json` . The subagent gets the tools; the parent conversation doesn't.

As of v2.1.153, the MCP restrictions that apply to the main session also cover servers declared in subagent frontmatter:

- `--strict-mcp-config` and `--bare`
- **Enterprise managed MCP configuration**
- **allowedMcpServers and deniedMcpServers policies**

When one of these blocks a server, Claude Code skips it and shows a warning naming the blocked servers.

Managed-settings restrictions apply to every subagent regardless of how it is defined. `--strict-mcp-config` doesn't filter servers you pass inline via `--agents` or the SDK `agents` option, since those are explicit caller input.

Permission modes

The `permissionMode` field controls how the subagent handles permission prompts. Subagents inherit the permission context from the main conversation and can override the mode, except when

the parent mode takes precedence as described below.

Mode	Behavior
default	Standard permission checking with prompts
acceptEdits	Auto-accept file edits and common filesystem commands for paths in the working directory or additionalDirectories
auto	Auto mode: a background classifier reviews commands and protected-directory writes
dontAsk	Auto-deny permission prompts (explicitly allowed tools still work)
bypassPermissions	Skip permission prompts
plan	Plan mode (read-only exploration)

⚠ Use `bypassPermissions` with caution. It skips permission prompts, allowing the subagent to execute operations without approval, including writes to `.git`, `.config/git`, `.claude`, `.vscode`, `.idea`, `.husky`, `.cargo`, `.devcontainer`, `.yarn`, and `.mvn`. Explicit `ask` **rules** and root and home directory removals such as `rm -rf /` still prompt. See [permission modes](#) for details.

If the parent uses `bypassPermissions` or `acceptEdits`, this takes precedence and can't be overridden. If the parent uses **auto mode**, the subagent inherits auto mode and any `permissionMode` in its frontmatter is ignored: the classifier evaluates the subagent's tool calls with the same block and allow rules as the parent session.

Preload skills into subagents

Use the `skills` field to inject skill content into a subagent's context at startup. This gives the subagent domain knowledge without requiring it to discover and load skills during execution.

```
---
name: api-developer
description: Implement API endpoints following team conventions
skills:
  - api-conventions
  - error-handling-patterns
---
```

Implement API endpoints. Follow the conventions and patterns from the preloaded skills.

The full content of each listed skill is injected into the subagent's context at startup. This field controls which skills are preloaded, not which skills the subagent can access: without it, the subagent can still discover and invoke project, user, and plugin skills through the Skill tool during execution. To prevent a subagent from invoking skills entirely, omit `Skill` from the `tools` list or add it to `disallowedTools`.

You can't preload skills that set `disable-model-invocation: true`, since preloading draws from the same set of skills Claude can invoke. If a listed skill is missing or disabled, Claude Code skips it and logs a warning to the debug log.

❗ This is the inverse of [running a skill in a subagent](#). With `skills` in a subagent, the subagent controls the system prompt and loads skill content. With `context: fork` in a skill, the skill content is injected into the agent you specify. Both use the same underlying system.

Enable persistent memory

The `memory` field gives the subagent a persistent directory that survives across conversations. The subagent uses this directory to build up knowledge over time, such as codebase patterns, debugging insights, and architectural decisions.

```
---
name: code-reviewer
description: Reviews code for quality and best practices
memory: user
---

You are a code reviewer. As you review code, update your agent
memory with
patterns, conventions, and recurring issues you discover.
```

Choose a scope based on how broadly the memory should apply:

Scope	Location	Use when
user	~/.claude/agent-memory/<name-of-agent>/	the subagent should remember learnings across all projects
project	.claude/agent-memory/<name-of-agent>/	the subagent's knowledge is project-specific and shareable via version control
local	.claude/agent-memory-local/<name-of-agent>/	the subagent's knowledge is project-specific but should not be checked into version control

When memory is enabled:

- The subagent's system prompt includes instructions for reading and writing to the memory directory.
- The subagent's system prompt also includes the first 200 lines or 25KB of `MEMORY.md` in the memory directory, whichever comes first, with instructions to curate `MEMORY.md` if it exceeds that limit.
- Read, Write, and Edit tools are automatically enabled so the subagent can manage its memory files.

Persistent memory tips

- `project` is the recommended default scope. It makes subagent knowledge shareable via version control. Use `user` when the subagent's knowledge is broadly applicable across projects, or `local` when the knowledge should not be checked into version control.
- Ask the subagent to consult its memory before starting work: "Review this PR, and check your memory for patterns you've seen before."
- Ask the subagent to update its memory after completing a task: "Now that you're done, save what you learned to your memory." Over time, this builds a knowledge base that makes the subagent more effective.
- Include memory instructions directly in the subagent's markdown file so it proactively maintains its own knowledge base:

Update your agent memory as you discover codepaths, patterns, library locations, and key architectural decisions. This builds up institutional knowledge across conversations. Write concise notes about what you found and where.

Conditional rules with hooks

For more dynamic control over tool usage, use `PreToolUse` hooks to validate operations before they execute. This is useful when you need to allow some operations of a tool while blocking others.

This example creates a subagent that only allows read-only database queries. The `PreToolUse` hook runs the script specified in `command` before each Bash command executes:

```
---
name: db-reader
description: Execute read-only database queries
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

Claude Code passes hook input as JSON via stdin to hook commands. The validation script reads this JSON, extracts the Bash command, and exits with code 2 to block write operations:

```
#!/bin/bash
# ./scripts/validate-readonly-query.sh

INPUT=$(cat)
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

# Block SQL write operations (case-insensitive)
if echo "$COMMAND" | grep -iE
'\b(INSERT|UPDATE|DELETE|DROP|CREATE|ALTER|TRUNCATE)\b' >
/dev/null; then
    echo "Blocked: Only SELECT queries are allowed" >&2
    exit 2
fi

exit 0
```

See [Hook input](#) for the complete input schema and [exit codes](#) for how exit codes affect behavior. On Windows, write hook scripts in PowerShell and add `shell: powershell` to the hook entry as shown in [running hooks in PowerShell](#).

Disable specific subagents

You can prevent Claude from using specific subagents by adding them to the `deny` array in your [settings](#). Use the format `Agent(subagent-name)` where `subagent-name` matches the subagent's name field.

```
{
  "permissions": {
    "deny": ["Agent(Explore)", "Agent(my-custom-agent)"]
  }
}
```

This works for both built-in and custom subagents. You can also use the `--disallowedTools` CLI flag:

```
claude --disallowedTools "Agent(Explore)"
```

See [Permissions documentation](#) for more details on permission rules.

Define hooks for subagents

Subagents can define hooks that run during the subagent's lifecycle. There are two ways to configure hooks:

- **In the subagent's frontmatter:** define hooks that run only while that subagent is active
- **In `settings.json` :** define hooks that run in the main session when subagents start or stop

Hooks in subagent frontmatter

Define hooks directly in the subagent's markdown file. These hooks only run while that specific subagent is active and are cleaned up when it finishes.

 Frontmatter hooks fire when the agent is spawned as a subagent through the Agent tool or an @-mention, and when the agent runs as the main session via `--agent` or the `agent` setting. In the main-session case they run alongside any hooks defined in `settings.json`.

All hook events are supported. The most common events for subagents are:

Event	Matcher input	When it fires
<code>PreToolUse</code>	Tool name	Before the subagent uses a tool
<code>PostToolUse</code>	Tool name	After the subagent uses a tool
<code>Stop</code>	(none)	When the subagent finishes (converted to <code>SubagentStop</code> at runtime)

This example validates Bash commands with the `PreToolUse` hook and runs a linter after file edits with `PostToolUse` :

```

---
name: code-reviewer
description: Review code changes with automatic linting
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-command.sh $TOOL_INPUT"
  PostToolUse:
    - matcher: "Edit|Write"
      hooks:
        - type: command
          command: "./scripts/run-linter.sh"
---

```

When the agent is invoked as a subagent, `Stop` hooks in frontmatter are automatically converted to `SubagentStop` events.

Project-level hooks for subagent events

Configure hooks in `settings.json` that respond to subagent lifecycle events in the main session.

Event	Matcher input	When it fires
<code>SubagentStart</code>	Agent type name	When a subagent begins execution
<code>SubagentStop</code>	Agent type name	When a subagent completes

Both events support matchers to target specific agent types by name. This example runs a setup script only when the `db-agent` subagent starts, and a cleanup script when any subagent stops:

```

{
  "hooks": {
    "SubagentStart": [
      {
        "matcher": "db-agent",
        "hooks": [
          { "type": "command", "command": "./scripts/setup-db-
connection.sh" }
        ]
      }
    ],
    "SubagentStop": [
      {
        "hooks": [
          { "type": "command", "command": "./scripts/cleanup-db-
connection.sh" }
        ]
      }
    ]
  }
}

```

See [Hooks](#) for the complete hook configuration format.

Work with subagents

Understand automatic delegation

Claude automatically delegates tasks based on the task description in your request, the `description` field in subagent configurations, and current context. To encourage proactive delegation, include phrases like “use proactively” in your subagent’s description field.

Invoke subagents explicitly

When automatic delegation isn’t enough, you can request a subagent yourself. Three patterns escalate from a one-off suggestion to a session-wide default:

- **Natural language:** name the subagent in your prompt; Claude decides whether to delegate
- **@-mention:** guarantees the subagent runs for one task
- **Session-wide:** the whole session uses that subagent’s system prompt, tool restrictions, and

model via the `--agent` flag or the `agent` setting

For natural language, there's no special syntax. Name the subagent and Claude typically delegates:

```
Use the test-runner subagent to fix failing tests
Have the code-reviewer subagent look at my recent changes
```

@-mention the subagent. Type `@` and pick the subagent from the typeahead, the same way you @-mention files. This ensures that specific subagent runs rather than leaving the choice to Claude:

```
@code-reviewer (agent) look at the auth changes
```

Your full message still goes to Claude, which writes the subagent's task prompt based on what you asked. The @-mention controls which subagent Claude invokes, not what prompt it receives.

Subagents provided by an enabled **plugin** appear in the typeahead under their scoped name, such as `my-plugin:code-reviewer` or `my-plugin:review:security` when the plugin **organizes agents into subfolders**. Named background subagents currently running in the session also appear in the typeahead, showing their status next to the name.

You can also type the mention manually without using the picker: `@agent-<name>` for local subagents, or `@agent-` followed by the scoped name for plugin subagents, for example `@agent-my-plugin:code-reviewer`.

Run the whole session as a subagent. Pass `--agent <name>` to start a session where the main thread itself takes on that subagent's system prompt, tool restrictions, and model:

```
claude --agent code-reviewer
```

The subagent's system prompt replaces the default Claude Code system prompt entirely, the same way `--system-prompt` does. `CLAUDE.md` files and project memory still load through the normal message flow. The agent name appears as `@<name>` in the startup header so you can confirm it's active.

This works with built-in and custom subagents, and the choice persists when you resume the session.

For a plugin-provided subagent, you can pass only the agent name and Claude Code finds it:

```
claude --agent security-reviewer
```

If multiple plugins provide agents with the same name, pass the scoped name to disambiguate:

```
claude --agent my-plugin:security-reviewer
```

If the plugin places the agent in a subfolder of its `agents/` directory, include the subfolder in the scoped name, for example `claude --agent my-plugin:review:security`.

To make it the default for every session in a project, set `agent` in `.claude/settings.json`:

```
{  
  "agent": "code-reviewer"  
}
```

The CLI flag overrides the setting if both are present.

Run subagents in foreground or background

Subagents can run in the foreground or the background:

- **Foreground subagents** block the main conversation until complete. Permission prompts are passed through to you as they come up.
- **Background subagents** run concurrently while you continue working. As of v2.1.186, when a background subagent reaches a tool call that needs permission, the prompt surfaces in your main session and names the subagent that is asking. Approve to let the subagent continue, or press Esc to deny that one tool call without stopping the subagent. Before v2.1.186, background subagents auto-denied any tool call that would have prompted.

Claude decides whether to run subagents in the foreground or background based on the task. You can also:

- Ask Claude to “run this in the background”
- Press **Ctrl+B** to background a running task

To disable all background task functionality, set the `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS` environment variable to `1`. See [Environment variables](#).

When `CLAUDE_CODE_FORK_SUBAGENT` is set to `1`, every subagent spawn runs in the background regardless of the `background` field. Permission prompts from these background subagents surface in your main session as described above.

Common patterns

Isolate high-volume operations

One of the most effective uses for subagents is isolating operations that produce large amounts of output. Running tests, fetching documentation, or processing log files can consume significant context. By delegating these to a subagent, the verbose output stays in the subagent's context while only the relevant summary returns to your main conversation.

```
Use a subagent to run the test suite and report only the failing
tests with their error messages
```

Run parallel research

For independent investigations, spawn multiple subagents to work simultaneously:

```
Research the authentication, database, and API modules in
parallel using separate subagents
```

Each subagent explores its area independently, then Claude synthesizes the findings. This works best when the research paths don't depend on each other.

 When subagents complete, their results return to your main conversation. Running many subagents that each return detailed results can consume significant context.

For tasks that need sustained parallelism or exceed your context window, **agent teams** give each worker its own independent context.

Chain subagents

For multi-step workflows, ask Claude to use subagents in sequence. Each subagent completes its task and returns results to Claude, which then passes relevant context to the next subagent.

Use the code-reviewer subagent to find performance issues, then use the optimizer subagent to fix them

Choose between subagents and main conversation

Use the **main conversation** when:

- The task needs frequent back-and-forth or iterative refinement
- Multiple phases share significant context, such as planning, implementation, and testing
- You're making a quick, targeted change
- Latency matters. Subagents start fresh and may need time to gather context

Use **subagents** when:

- The task produces verbose output you don't need in your main context
- You want to enforce specific tool restrictions or permissions
- The work is self-contained and can return a summary

Consider **Skills** instead when you want reusable prompts or workflows that run in the main conversation context rather than isolated subagent context.

For a quick question about something already in your conversation, use `/btw` instead of a subagent. It sees your full context but has no tool access, and the answer is discarded rather than added to history.

Spawn nested subagents

As of Claude Code v2.1.172, a subagent can spawn its own subagents. Use this when a delegated task itself splits into parallel subtasks, such as a reviewer subagent that dispatches a verifier per finding, so the intermediate output never reaches your main conversation. Only the top-level subagent's summary returns to you.

A nested subagent is configured the same way as a top-level one and resolves from the same **scopes**. The subagent panel below the prompt input shows the full tree: each row displays a `(+N)` count of descendants, and opening a row shows that subagent's direct children with a path back to `main`. The Running tab in `/agents` lists running subagents as a flat list.

Depth is counted as the number of subagent levels below the main conversation, regardless of

whether each level runs in the foreground or background. A subagent at depth five doesn't receive the Agent tool and can't spawn further. The limit is fixed and not configurable.

As of Claude Code v2.1.187, a background subagent's depth is fixed when it is first spawned, and resuming it later doesn't change that depth. For example, if your main conversation spawns subagent A, and A spawns a background subagent B at depth two, B is still at depth two when you resume it directly from the main conversation. Resuming a subagent from a shallower context doesn't let it spawn additional levels that the depth limit already prevented.

To prevent a specific subagent from spawning others, omit `Agent` from its tools list or add it to `disallowedTools` .

A fork still can't spawn another fork. It can spawn other subagent types, and those count toward the depth limit.

Manage subagent context

What loads at startup

Each subagent starts with a fresh, isolated context window. It doesn't see your conversation history, the skills you've already invoked, or the files Claude has already read. Claude composes a delegation message that summarizes the task, and the subagent works from there. The exception is a fork, which inherits the parent conversation instead of starting fresh.

A non-fork subagent's initial context contains:

- **System prompt:** the agent's own prompt plus environment details that Claude Code appends, not the full Claude Code system prompt. Custom subagents define theirs in the markdown body or `prompt` field. Built-in agents have predefined prompts.
- **Task message:** the delegation prompt Claude writes when it hands off the work.
- **CLAUDE.md and memory:** every level of the memory hierarchy the main conversation loads, including `~/ .claude/CLAUDE.md` , project rules, `CLAUDE.local.md` , and managed policy files. The built-in Explore and Plan agents skip this.
- **Git status:** a snapshot taken at the start of the parent session. Absent when the working directory isn't a Git repository or when `includeGitInstructions` is `false` . Explore and Plan skip it regardless.
- **Preloaded skills:** full content of any skill named in the agent's `skills` field. Built-in agents don't preload skills.

Explore and Plan are the only subagents that omit CLAUDE.md and git status. There is no frontmatter field or per-agent setting to change which agents skip them.

The main conversation reads Explore and Plan results with full CLAUDE.md context, so most rules don't need to reach the subagent itself. If a rule must, such as "ignore the `vendor/` directory," restate it in the prompt you give Claude when delegating.

Resume subagents

Each subagent invocation creates a new instance with fresh context. To continue an existing subagent's work instead of starting over, ask Claude to resume it.

Resumed subagents retain their full conversation history, including all previous tool calls, results, and reasoning. The subagent picks up exactly where it stopped rather than starting fresh.

When a subagent completes, Claude receives its agent ID. The built-in Explore and Plan agents are one-shot and return no agent ID, so they can't be resumed; use `general-purpose` or a custom subagent when you need to continue the work.

Claude uses the `SendMessage` tool with the agent's ID as the `to` field to resume it. The `SendMessage` tool is always available for resuming subagents by agent ID or name. Structured team-protocol messages such as `shutdown_request` and `plan_approval_response` require **agent teams** to be enabled.

To resume a subagent, ask Claude to continue the previous work:

```
Use the code-reviewer subagent to review the authentication
module
```

```
[Agent completes]
```

```
Continue that code review and now analyze the authorization logic
```

```
[Claude resumes the subagent with full context from previous
conversation]
```

If a stopped subagent receives a `SendMessage`, it auto-resumes in the background without requiring a new `Agent` invocation.

You can also ask Claude for the agent ID if you want to reference it explicitly, or find IDs in the transcript files at `~/.claude/projects/{project}/{sessionId}/subagents/`. Each transcript is stored as `agent-{agentId}.jsonl`.

Subagent transcripts persist independently of the main conversation:

- **Main conversation compaction:** when the main conversation compacts, subagent transcripts are unaffected. They're stored in separate files.
- **Session persistence:** subagent transcripts persist within their session. You can resume a subagent after restarting Claude Code by resuming the same session.
- **Automatic cleanup:** transcripts are cleaned up based on the `cleanupPeriodDays` setting, which defaults to 30 days.

Auto-compaction

Subagents support automatic compaction using the same logic as the main conversation.

Compaction triggers under the same conditions, and `CLAUDE_AUTOCOMPACT_PCT_OVERRIDE` applies to subagents as well. See environment variables for when the override takes effect.

Compaction events are logged in subagent transcript files:

```
{
  "type": "system",
  "subtype": "compact_boundary",
  "compactMetadata": {
    "trigger": "auto",
    "preTokens": 167189
  }
}
```

The `preTokens` value shows how many tokens were used before compaction occurred.

Fork the current conversation

- ❗ Forked subagents require Claude Code v2.1.117 or later. From v2.1.161 the `/fork` command is enabled by default; on earlier versions it requires setting the `CLAUDE_CODE_FORK_SUBAGENT` environment variable to `1`. Letting Claude itself spawn forks is experimental and may change in future releases. This capability may also be enabled in interactive sessions as part of a staged rollout.

A fork is a subagent that inherits the entire conversation so far instead of starting fresh. This drops the input isolation that subagents otherwise provide: a fork sees the same system prompt, tools, model, and message history as the main session, so you can hand it a side task without re-explaining the situation. The fork's own tool calls still stay out of your conversation and only its final result

comes back, so your main context window stays clean. Use a fork when a named subagent would need too much background to be useful, or when you want to try several approaches in parallel from the same starting point.

To control fork mode regardless of the staged rollout, set `CLAUDE_CODE_FORK_SUBAGENT` to `1` to enable it explicitly or to `0` to disable it. The variable is honored in interactive mode and via the SDK or `claude -p`.

Enabling fork mode changes Claude Code in two ways:

- Claude can spawn a fork by requesting the `fork` subagent type explicitly. Spawns without a subagent type still use the general-purpose subagent, and named subagents such as Explore still spawn as before.
- Every subagent spawn runs in the background, whether it is a fork or a named subagent. Set `CLAUDE_CODE_DISABLE_BACKGROUND_TASKS` to `1` to keep spawns synchronous.

You can start a fork yourself with `/fork` followed by a directive, with or without the variable set. Claude Code names the fork from the first words of the directive. The following example forks the conversation to draft test cases while you continue with the implementation in the main session:

```
/fork draft unit tests for the parser changes so far
```

The fork appears in a panel below your prompt and runs in the background while you keep working. When it finishes, its result arrives as a message in your main conversation. The next section covers the panel controls for watching and steering forks while they run.

Observe and steer running forks

Running forks appear in a panel below the prompt input, with one row for the main session and one for each fork. Use these keys to interact with the panel:

Key	Action
↑ / ↓	Move between rows
Enter	Open the selected fork’s transcript and send it follow-up messages
x	Dismiss a finished fork or stop a running one
Esc	Return focus to the prompt input

How forks differ from named subagents

A fork inherits everything the main session has at the moment it spawns. A named subagent starts from its own definition.

	Fork	Named subagent
Context	Full conversation history	Fresh context with the prompt you pass
System prompt and tools	Same as main session	From the subagent's <u>definition file</u>
Model	Same as main session	From the subagent's <code>model</code> field
Permissions	Prompts surface in your terminal	<u>Prompts surface in your main session</u> when running in the background
Prompt cache	Shared with main session	Separate cache

Because a fork’s system prompt and tool definitions are identical to the parent, its first request reuses the parent’s prompt cache. This makes forking cheaper than spawning a fresh subagent for tasks that need the same context.

When Claude spawns a fork through the Agent tool, it can pass `isolation: "worktree"` so the fork’s file edits are written to a separate git worktree instead of your checkout.

Limitations

Setting `CLAUDE_CODE_FORK_SUBAGENT=1` enables fork mode in interactive sessions, non-interactive mode, and the Agent SDK; setting it to `0` disables fork mode everywhere, including any server-side rollout. A fork can’t spawn further forks.

Example subagents

These examples demonstrate effective patterns for building subagents. Use them as starting points, or generate a customized version with Claude.



Best practices:

- **Design focused subagents:** each subagent should excel at one specific task
- **Write detailed descriptions:** Claude uses the description to decide when to delegate
- **Limit tool access:** grant only necessary permissions for security and focus
- **Check into version control:** share project subagents with your team

Code reviewer

A read-only subagent that reviews code without modifying it. This example shows how to design a focused subagent with limited tool access (no Edit or Write) and a detailed prompt that specifies exactly what to look for and how to format output.

```
---
name: code-reviewer
description: Expert code review specialist. Proactively reviews
code for quality, security, and maintainability. Use immediately
after writing or modifying code.
tools: Read, Grep, Glob, Bash
model: inherit
---
```

You are a senior code reviewer ensuring high standards of code quality and security.

When invoked:

1. Run git diff to see recent changes
2. Focus on modified files
3. Begin review immediately

Review checklist:

- Code is clear and readable
- Functions and variables are well-named
- No duplicated code
- Proper error handling
- No exposed secrets or API keys
- Input validation implemented
- Good test coverage
- Performance considerations addressed

Provide feedback organized by priority:

- Critical issues (must fix)
- Warnings (should fix)
- Suggestions (consider improving)

Include specific examples of how to fix issues.

Debugger

A subagent that can both analyze and fix issues. Unlike the code reviewer, this one includes Edit because fixing bugs requires modifying code. The prompt provides a clear workflow from diagnosis to verification.

```
---
name: debugger
description: Debugging specialist for errors, test failures, and
unexpected behavior. Use proactively when encountering any
issues.
tools: Read, Edit, Bash, Grep, Glob
---
```

You are an expert debugger specializing in root cause analysis.

When invoked:

1. Capture error message and stack trace
2. Identify reproduction steps
3. Isolate the failure location
4. Implement minimal fix
5. Verify solution works

Debugging process:

- Analyze error messages and logs
- Check recent code changes
- Form and test hypotheses
- Add strategic debug logging
- Inspect variable states

For each issue, provide:

- Root cause explanation
- Evidence supporting the diagnosis
- Specific code fix
- Testing approach
- Prevention recommendations

Focus on fixing the underlying issue, not the symptoms.

Data scientist

A domain-specific subagent for data analysis work. This example shows how to create subagents for specialized workflows outside of typical coding tasks. It explicitly sets `model: sonnet` for more capable analysis.

```
---
name: data-scientist
description: Data analysis expert for SQL queries, BigQuery
operations, and data insights. Use proactively for data analysis
tasks and queries.
tools: Bash, Read, Write
model: sonnet
---
```

You are a data scientist specializing in SQL and BigQuery analysis.

When invoked:

1. Understand the data analysis requirement
2. Write efficient SQL queries
3. Use BigQuery command line tools (bq) when appropriate
4. Analyze and summarize results
5. Present findings clearly

Key practices:

- Write optimized SQL queries with proper filters
- Use appropriate aggregations and joins
- Include comments explaining complex logic
- Format results for readability
- Provide data-driven recommendations

For each analysis:

- Explain the query approach
- Document any assumptions
- Highlight key findings
- Suggest next steps based on data

Always ensure queries are efficient and cost-effective.

Database query validator

A subagent that allows Bash access but validates commands to permit only read-only SQL queries. This example shows how to use `PreToolUse` hooks for conditional validation when you need finer control than the `tools` field provides.

```
---
name: db-reader
description: Execute read-only database queries. Use when
analyzing data or generating reports.
tools: Bash
hooks:
  PreToolUse:
    - matcher: "Bash"
      hooks:
        - type: command
          command: "./scripts/validate-readonly-query.sh"
---
```

You are a database analyst with read-only access. Execute SELECT queries to answer questions about the data.

When asked to analyze data:

1. Identify which tables contain the relevant data
2. Write efficient SELECT queries with appropriate filters
3. Present results clearly with context

You cannot modify data. If asked to INSERT, UPDATE, DELETE, or modify schema, explain that you only have read access.

Claude Code **passes hook input as JSON** via stdin to hook commands. The validation script reads this JSON, extracts the command being executed, and checks it against a list of SQL write operations. If a write operation is detected, the script **exits with code 2** to block execution and returns an error message to Claude via stderr.

Create the validation script anywhere in your project. The path must match the `command` field in your hook configuration:

```
#!/bin/bash
# Blocks SQL write operations, allows SELECT queries

# Read JSON input from stdin
INPUT=$(cat)

# Extract the command field from tool_input using jq
COMMAND=$(echo "$INPUT" | jq -r '.tool_input.command // empty')

if [ -z "$COMMAND" ]; then
    exit 0
fi

# Block write operations (case-insensitive)
if echo "$COMMAND" | grep -iE '\b(INsert|UPdate|DElete|DRop|CREate|ALTER|TRUNCATE|REPLACE|MERGE)\b' > /dev/null; then
    echo "Blocked: Write operations not allowed. Use SELECT queries only." >&2
    exit 2
fi

exit 0
```

On macOS and Linux, make the script executable:

```
chmod +x ./scripts/validate-readonly-query.sh
```

On Windows, write the validation script in PowerShell and add `shell: powershell` to the hook entry. See [running hooks in PowerShell](#).

The hook receives JSON via stdin with the Bash command in `tool_input.command`. Exit code 2 blocks the operation and feeds the error message back to Claude. See [Hooks](#) for details on exit codes and [Hook input](#) for the complete input schema.

Next steps

Now that you understand subagents, explore these related features:

- [Distribute subagents with plugins](#) to share subagents across teams or projects

- Run Claude Code programmatically with the Agent SDK for CI/CD and automation
- Use MCP servers to give subagents access to external tools and data